

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 1: Desarrollo de aplicaciones web en el entorno servidor

Lección 3: Arquitecturas Web

Concepto de Arquitectura Web

El Modelo de Capas

Plataformas para el Desarrollo en las Capas Servidor

Herramientas de Desarrollo Orientadas a Servidor de Aplicaciones Web

Concepto de Arquitectura Web

La arquitectura web **define la estructura y organización de una aplicación web**, determinando cómo interactúan sus componentes. Se basa en la separación de responsabilidades para mejorar la escalabilidad, mantenimiento y seguridad.

El Modelo de Capas

El modelo de capas segmenta la aplicación en distintos niveles con funciones específicas:

- **Capa de Presentación:** Interfaz de usuario (HTML, CSS, JavaScript).
- **Capa de Negocio:** Lógica de la aplicación (PHP, Node.js, Java).
- **Capa de Datos:** Base de datos y su gestión (MySQL, PostgreSQL, MongoDB).

Plataformas para el Desarrollo en las Capas Servidor

Existen diversas plataformas para desarrollar aplicaciones en servidor, como PHP, Python (Django, Flask), Node.js, Ruby on Rails y Java (Spring, JSP).

Herramientas de Desarrollo Orientadas a Servidor de Aplicaciones Web

1. Tipos de Herramientas

- **Entornos de Desarrollo Integrado (IDE):** PHPStorm, Visual Studio Code, NetBeans.
- **Servidores Web:** Apache, Nginx.
- **Bases de Datos:** MySQL, PostgreSQL.
- **Frameworks:** Laravel (PHP), Express.js (Node.js).

2. Extensibilidad e Instalación de Módulos

Los servidores y lenguajes permiten ampliar funcionalidades mediante módulos y extensiones:

Ejemplo: Instalar la extensión php-xml en Ubuntu/Debian

```
sudo apt install php-xml
```

Esto instala la extensión XML, que es necesaria para manipular archivos XML en PHP.

3. Configuración de Entornos de Desarrollo, Preproducción y Producción

Cada entorno tiene sus propias necesidades:

- **Desarrollo:** Mostrar todos los errores, usar una base de datos local y habilitar logs detallados.
- **Preproducción (staging):** Similar a producción, pero con debugging opcional y logs activos.
- **Producción:** Desactivar la visualización de errores, usar caching y optimizar el rendimiento.

Ejemplo Completo de Configuración por Entorno en PHP

Podemos manejar la configuración mediante un archivo .env (con vlucas/phpdotenv) o mediante constantes en un archivo config.php. Aquí un ejemplo usando config.php:

```
// Definir el entorno actual: 'development', 'staging' o 'production'  
define('APP_ENV', 'development'); // Cambiar según el entorno
```

```
// Configuración según el entorno
```

```
switch (APP_ENV) {  
    case 'development':  
        error_reporting(E_ALL);  
        ini_set('display_errors', 1);  
        define('DB_HOST', 'localhost');  
        define('DB_USER', 'root');  
        define('DB_PASS', 'root');  
        define('DB_NAME', 'mi_base_datos_dev');  
        break;  
  
    case 'staging':  
        error_reporting(E_ALL & ~E_NOTICE);  
        ini_set('display_errors', 0);  
        define('DB_HOST', 'staging-db.miweb.com');  
        define('DB_USER', 'staging_user');  
        define('DB_PASS', 'staging_pass');  
        define('DB_NAME', 'mi_base_datos_staging');
```

```

        break;

    case 'production':
        error_reporting(0);
        ini_set('display_errors', 0);
        define('DB_HOST', 'prod-db.miweb.com');
        define('DB_USER', 'prod_user');
        define('DB_PASS', 'prod_pass');
        define('DB_NAME', 'mi_base_datos_prod');
        break;

    default:
        die('Entorno no configurado correctamente.');
```

```

}
```

```

// Conectar a la base de datos con la configuración del entorno actual
```

```

try {
    $pdo = new PDO("mysql:host=" . DB_HOST . ";dbname=" . DB_NAME, DB_USER, DB_PASS);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Conexión exitosa al entorno: " . APP_ENV;
} catch (PDOException $e) {
    die("Error de conexión: " . $e->getMessage());
}
```

Explicación del Código

1. Definimos el entorno actual con APP_ENV.
2. Usamos switch para cambiar la configuración según el entorno.
3. Activamos o desactivamos errores con error_reporting() e ini_set('display_errors', 0).
4. Definimos las credenciales de la base de datos según el entorno.
5. Manejamos la conexión a MySQL con PDO, usando las credenciales correspondientes.

Cómo Usarlo

- Para cambiar de entorno, simplemente modifica APP_ENV en config.php.

- Si quieres usar un archivo .env, puedes utilizar la librería vlucas/phpdotenv.
- En producción, recuerda desactivar `display_errors` para evitar mostrar información sensible.

4. Funcionalidades de Depuración

El uso de herramientas como Xdebug en PHP permite identificar errores y mejorar la calidad del código.